Recordatorio de tiempos

- **Tiempo de transmisión (T_t):** es el tiempo que tarda el paquete en ser emitido por la interfaz:

  $T_t = p / v$

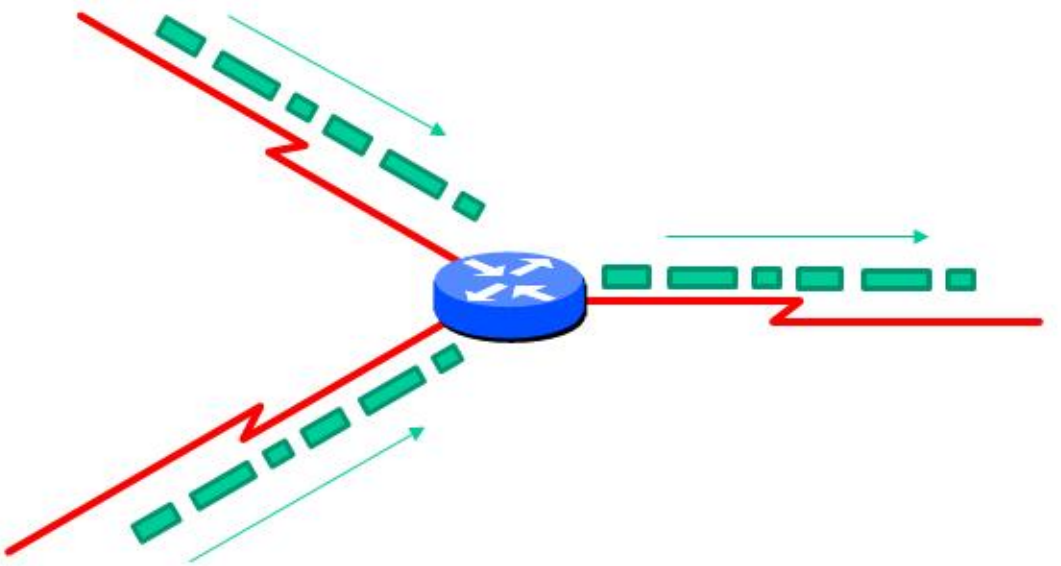
  p = tamaño del paquete (en bits)

  v = velocidad de la línea (en bits/s)

- **Tiempo de propagación (τ):** es el tiempo que tarda un bit en viajar desde la interfaz de origen a la de destino. Depende de la distancia y del tipo de medio (cobre, fibra o aire)
- **Tiempo de entrega o latencia (T_e):** es la suma del tiempo de transmisión y el de propagación: $T_e = T_t + \tau$

Tiempo de espera

- **Tiempo de espera (T_{esp}):** es el tiempo que el paquete espera haciendo cola en la interfaz de entrada del router hasta que le llega su turno. El tiempo de espera (T_{esp}) depende del grado de congestión en la interfaz de salida.



Tiempo de servicio

- **Tiempo de servicio ($T_s = T_{esp} + T_t$):** es el tiempo que transcurre desde que el paquete llega por la interfaz de entrada (se pone en cola) hasta que se emite por la interfaz de salida. No incluye el tiempo propagación τ (tiempo de llegar al siguiente router).
- Por teoría de colas puede demostrarse que:

  $$T_s = p / (v - c)$$

  p = tamaño del paquete (en bits)

  v = velocidad nominal (o capacidad) de la línea (en bits/s)

  c = tráfico o caudal real de la línea (en bits/s)

Ej.: paquete de 500 bytes en una línea de 64 Kb/s al 50% de ocupación:

p = 4.000, v = 64.000, c = 32.000 $\rightarrow>$ T_s = 125 ms

Propiedades del Tiempo de servicio

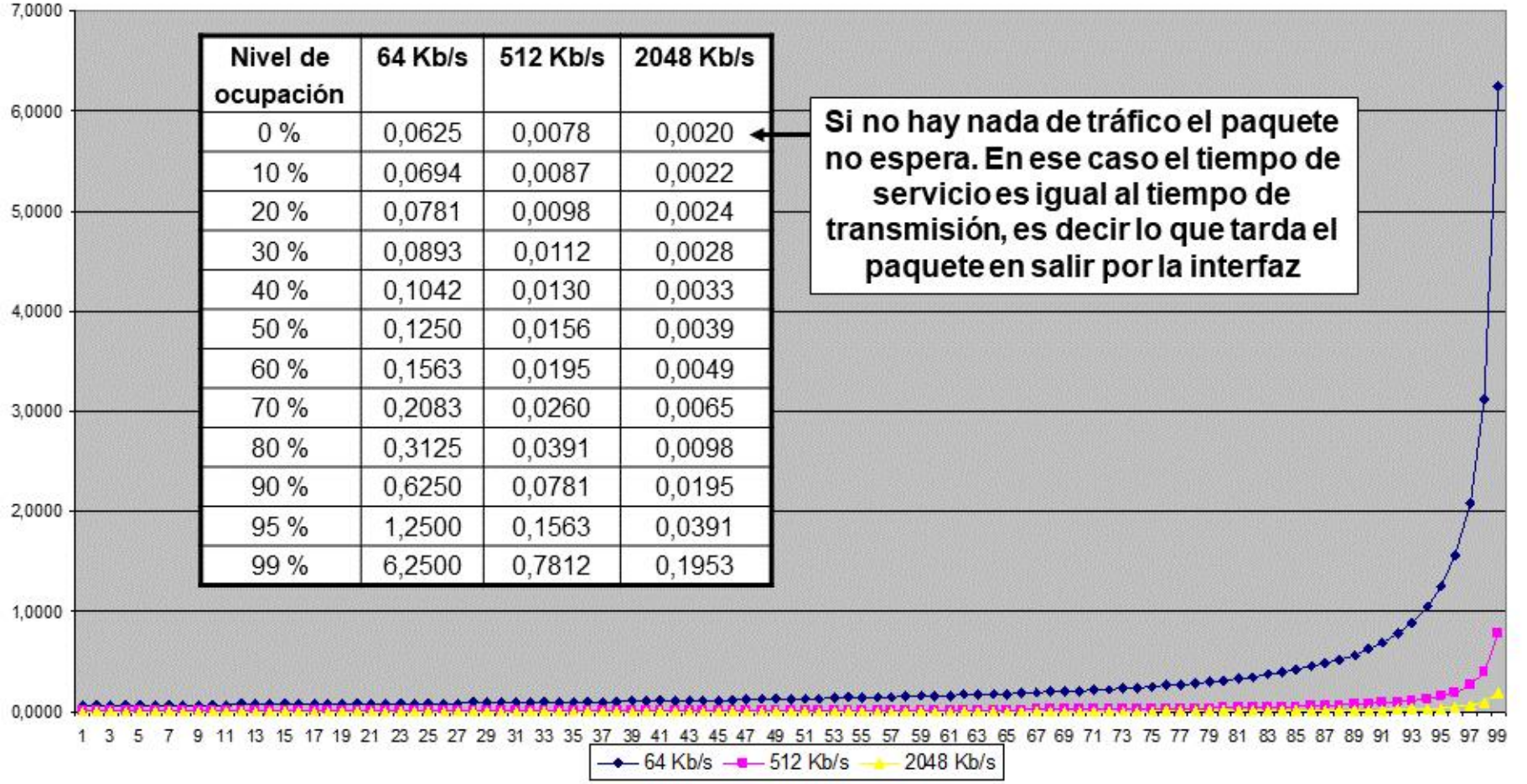
$$T_s = p / (v - c) \qquad T_t = p / v$$

- Si c = 0 (línea sin tráfico) entonces $T_s = p / v = T_t$
- El T_s de una interfaz de velocidad *v* y caudal *c* es igual al de una línea sin tráfico de velocidad v-c, ejemplos:
  - Una línea de 64 Kb/s al 50% de ocupación tiene el mismo T_s que una línea de 32 Kb/s sin tráfico
  - Una Ethernet 100BASE-T al 90% de ocupación tiene el mismo T_s que una ethernet 10BASE-T sin tráfico
- Cuando la velocidad de una línea (v) es elevada podemos llegar a niveles altos de ocupación sin incurrir en T_s excesivos
- Si el tamaño de los paquetes aumenta el T_s aumenta

T_s para p=500 bytes y v=64, 512 y 2048 Kb/s

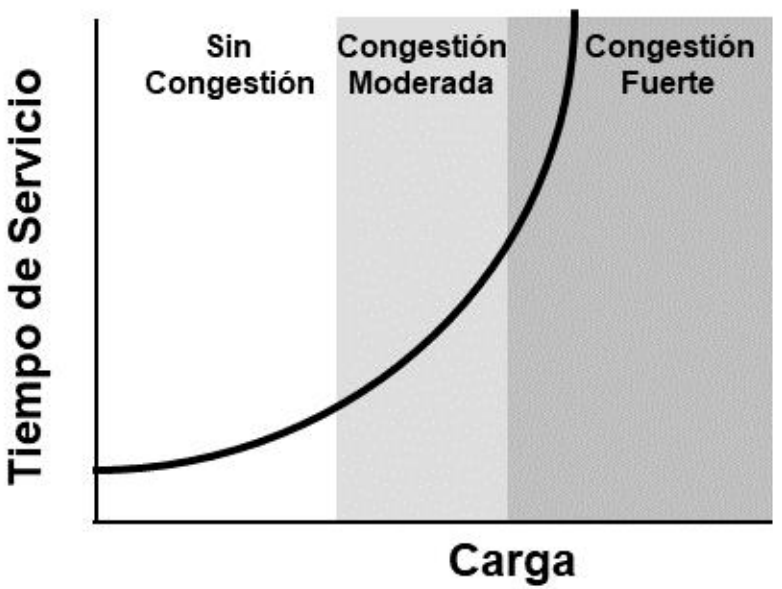
Nivel de ocupación	64 Kb/s	512 Kb/s	2048 Kb/s
0 %	0,0625	0,0078	0,0020
10 %	0,0694	0,0087	0,0022
20 %	0,0781	0,0098	0,0024
30 %	0,0893	0,0112	0,0028
40 %	0,1042	0,0130	0,0033
50 %	0,1250	0,0156	0,0039
60 %	0,1563	0,0195	0,0049
70 %	0,2083	0,0260	0,0065
80 %	0,3125	0,0391	0,0098
90 %	0,6250	0,0781	0,0195
95 %	1,2500	0,1563	0,0391
99 %	6,2500	0,7812	0,1953

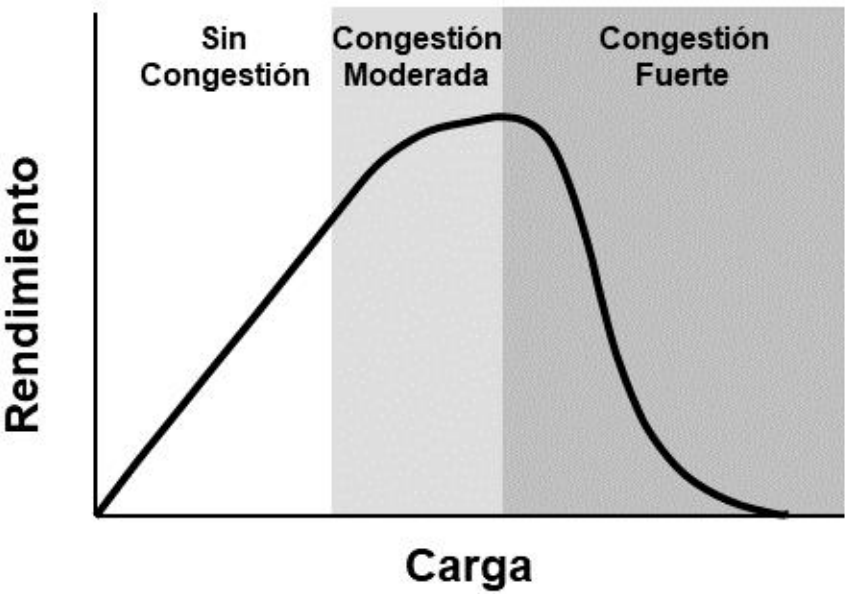
Si no hay nada de tráfico el paquete no espera. En ese caso el tiempo de servicio es igual al tiempo de transmisión, es decir lo que tarda el paquete en salir por la interfaz



Las líneas de baja velocidad sufren mayores retardos cuando hay congestión

Efecto de la ocupación de un enlace en el tiempo de servicio y el rendimiento





Con cargas elevadas los paquetes se pierden y han de reenviarse, por lo que por encima de un nivel de congestión el rendimiento disminuye

Diseño de capacidad de una red

- Desde un punto de vista de optimizar recursos lo ideal es que los enlaces tengan un nivel de congestión moderada
- Una congestión excesiva provoca la pérdida y reenvío de paquetes, el fallo de protocolos de alto nivel y la frustración de los usuarios
- La ausencia de congestión indica un sobredimensionamiento, lo cual podría significar un derroche de recursos (salvo que estos sean muy baratos, ej. LANs)
- Hay que tener en cuenta las necesidades y la evolución de tráfico a lo largo del día; lo razonable es dimensionar tomando en cuenta las necesidades en horas punta

Tiempo de servicio con velocidades o caudales asimétricos

- Cuando el enlace es asimétrico (diferente velocidad en cada sentido, como ADSL) el tiempo de servicio será diferente para cada sentido
- Aun cuando el enlace sea simétrico el tiempo de servicio en general será diferente, ya que casi siempre el caudal es diferente para cada sentido. Por tanto los cálculos deben hacerse para cada sentido por separado

Tiempo de servicio de una ruta

- Cuando un paquete ha de pasar por más de un enlace el tiempo de servicio de la ruta es la suma de los tiempos de servicio de los enlaces que atraviesa

Ejercicio sobre T_s

- Una empresa, con sedes en Valencia y Milán, quiere utilizar un enlace de 512 Kb/s para datos y para telefonía IP. Según la norma ITU G.114 el retardo máximo admisible en telefonía para que la calidad sea aceptable es de 150 ms.
- Calcular cual será el caudal máximo para que se pueda mantener conversaciones cumpliendo ese requisito, y cual será el número máximo de conversaciones simultáneas suponiendo que no haya otro tráfico en la línea.
- Datos adicionales:
  - Cada comunicación telefónica ocupa 80 Kb/s
  - Suponga paquetes de 500 bytes
  - Tiempo de propagación τ Valencia-Milán: 20 ms.

Solución

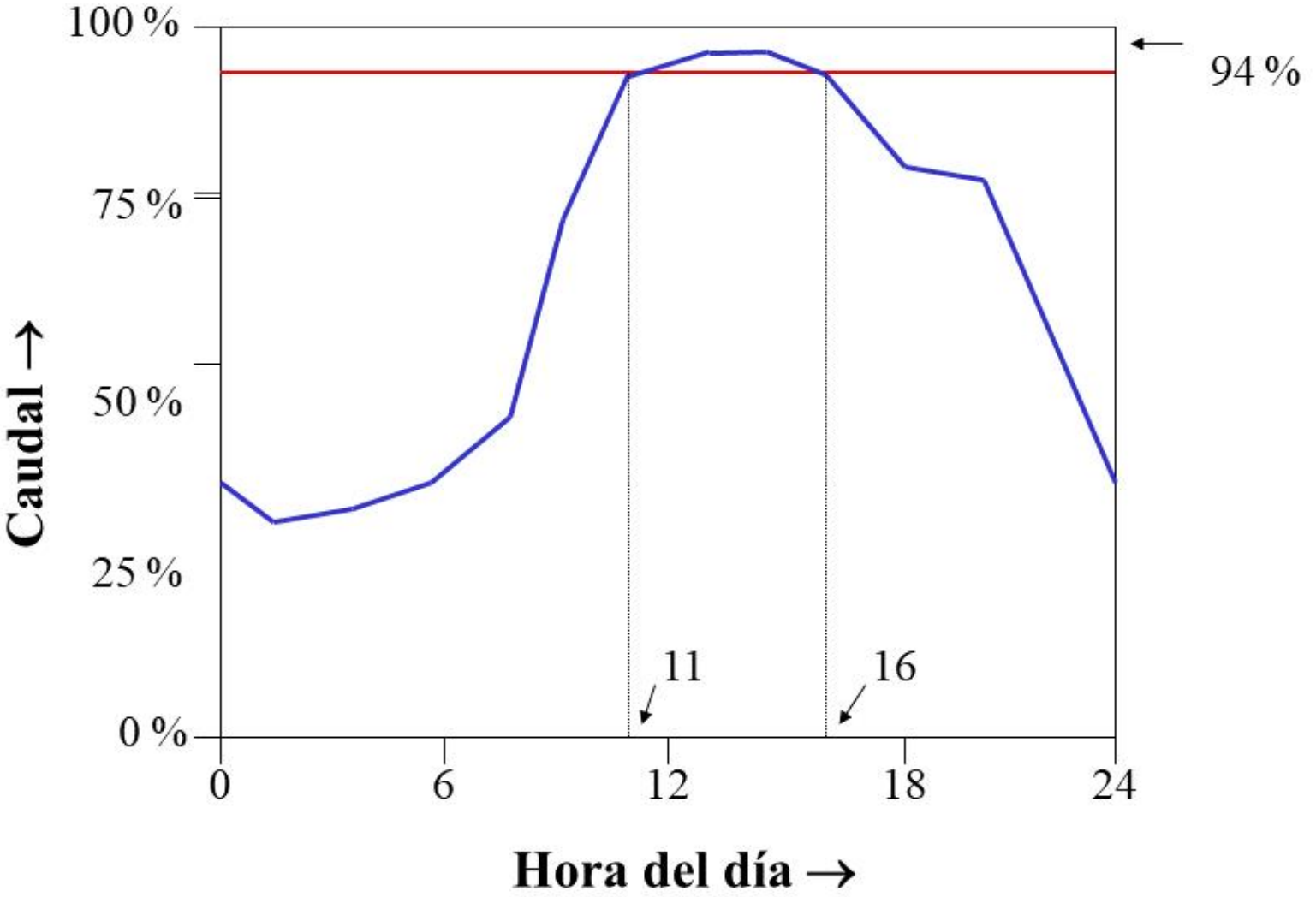
- El límite máximo será el caudal que nos dé T_s = 130 ms, ya que un caudal mayor nos dará un retardo total ($T_s + \tau$) superior a 150 ms
- De $\mathbf{T_s = p / (v - c)}$ despejamos c y ponemos valores:

  $c = v - (p/T_s) = 512.000 - (4.000 / 0{,}13) = 481.000$
- Por tanto empezaremos a tener problemas cuando la línea tenga una ocupación superior a 481 Kb/s (94%). Dentro de este límite nos cabrían 6 conversaciones simultáneas (80 x 6 = 480 Kb/s, suponiendo que no hay otro tráfico en la línea)
- Con una línea de 2 Mb/s sería:

  $c = v - (p/T_s) = 2.048.000 - (4.000 / 0{,}13) = 2.017.000$

  2.017 Kb/s corresponde a una ocupación del 98,5%

Perfil de tráfico

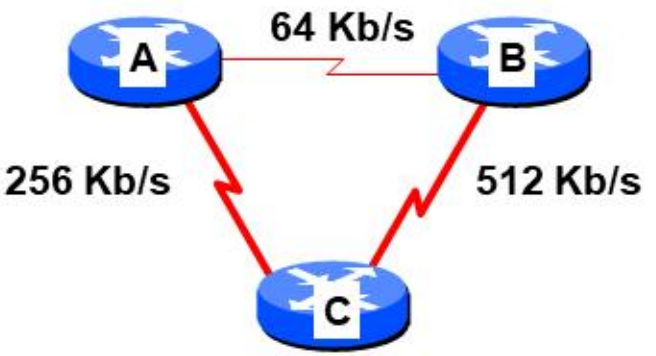


Ejemplo de routing estático: routing basado en el flujo

- Un buen criterio para elegir la ruta óptima es el **tiempo de servicio (T_s)** promedio. La ruta con un T_s promedio más bajo se considera mejor
- Requiere disponer de estimaciones de tráfico entre cada par de nodos de la red (matriz de tráfico).
- Se plantean varias topologías (todas las posibles o solo las que por heurística se consideran más interesantes), se realiza la simulación para cada una, se comparan sus T_s promedio y se elige la mejor.

Routing basado en el flujo: ejemplo

- Supongamos una red como la siguiente:

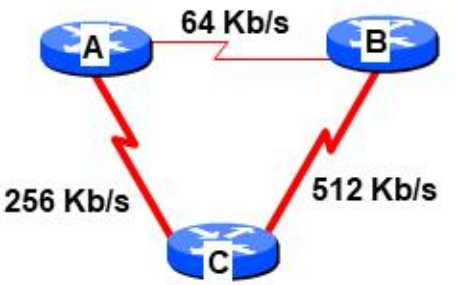


- Es obvio que el tráfico A-C o B-C debe ir por el enlace directo. Pero ¿que ocurre con el tráfico A-B? ¿Deberá ir por el enlace directo, o quizá será mejor que vaya a través de C? Vamos a verlo

Routing basado en el flujo: ejemplo

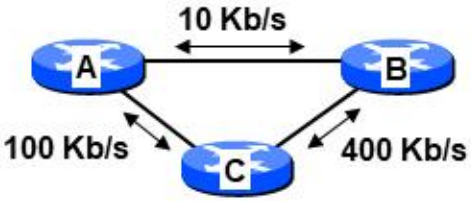
Matriz de tráfico (Kb/s)

Origen \ Destino	A	B	C
A	-	10	100
B	10	-	400
C	100	400	-



- Con la matriz de tráfico podemos calcular el caudal de cada enlace para los dos supuestos planteados (ruta A-B directa y ruta A-C-B indirecta). Y con eso calculamos las matriz de los T_s (que no es mas que el T_s para cada enlace)

Ruta directa (A-B)



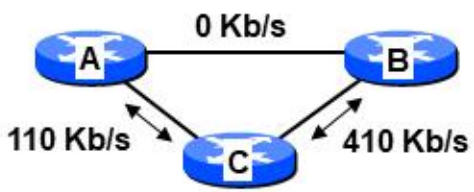
Enlace	Velocidad	Caudal	Ocupación	T_s
AB	64 Kb/s	10 Kb/s	15,6 %	74,1 ms
AC	256 Kb/s	100 Kb/s	39,1 %	25,6 ms
BC	512 Kb/s	400 Kb/s	78,1 %	35,7 ms

Matriz de T_s

	A	B	C
A	-	74,1	25,6
B	74,1	-	35,7
C	25,6	35,7	-

T_s promedio: 45,1 ms

Ruta indirecta (A-C-B)



Enlace	Velocidad	Caudal	Ocupación	T_s
AB	64 Kb/s	0 Kb/s	0 %	62,5 ms
AC	256 Kb/s	110 Kb/s	43,0 %	27,4 ms
BC	512 Kb/s	410 Kb/s	80,1 %	39,2 ms

Matriz de T_s

	A	B	C
A	-	**66,6**	27,4
B	**66,6**	-	39,2
C	27,4	39,2	-

27,4 ms + 39,2 ms = 66,6 ms

T_s promedio: **44,4** ms

T_s promedio vs T_s promedio ponderado

- Atendiendo al T_s promedio la ruta indirecta es mejor. Pero la media aritmética de los T_s da la misma importancia a todos los enlaces, independientemente de su tráfico
- Para una correcta valoración hay que calcular la media ponderada atendiendo al tráfico de cada enlace

Cálculo del T_s medio ponderado

Matriz de tráfico (Kb/s)

Destino

Origen	A	B	C
A	-	10	100
B	10	-	400
C	100	400	-

Tráfico total: 1020 Kb/s

X 1/1020

M_{norm}: Matriz de tráfico normalizada

	A	B	C
A	-	0,0098	0,0980
B	0,0098	-	0,3922
C	0,0980	0,3922	-

100/1020 = 0,098

Ruta A-B:

T_s

	A	B	C
A	-	74,1	25,6
B	74,1	-	35,7
C	25,6	35,7	-

T_s medio: 45,1 ms

X M_{norm}

T_s ponderados

	A	B	C
A	-	0,726	2,509
B	0,726	-	14,002
C	2,509	14,002	-

25,6*0,098 = 2,509

T_s medio ponderado: 34,5 ms

Ruta A-C-B:

T_s

	A	B	C
A	-	66,6	27,4
B	66,6	-	39,2
C	27,4	39,2	-

T_s medio: 44,4 ms

X M_{norm}

T_s ponderados

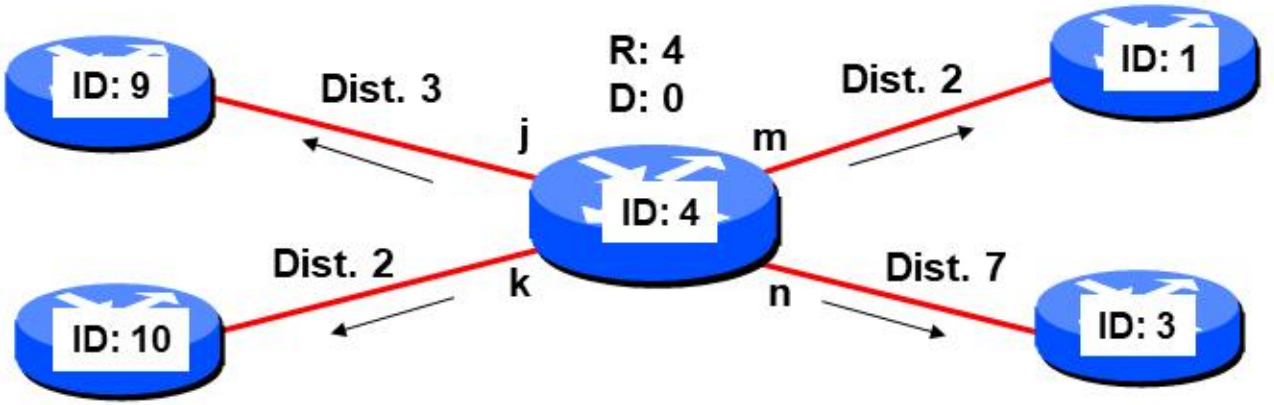
	A	B	C
A	-	0,653	2,685
B	0,653	-	15,374
C	2,685	15,374	-

27,4*0,098 = 2,685

T_s medio ponderado: 37,4 ms

Conclusión: La ruta A-B es mejor que la A-C-B

Ejemplo del algoritmo vector distancia



Destino:	1	2	3	4	5	6	7	8	9	10	11
Recibido de j (+3):	12	3	15	3	12	5	6	18	0	7	15
Recibido de k (+2):	5	8	3	2	10	7	4	20	5	0	15
Recibido de m (+2):	0	5	3	2	19	9	5	22	2	4	7
Recibido de n (+7):	6	2	0	7	8	5	8	12	11	3	2
Distancia mínima:	2	6	5	0	12	8	6	19	3	2	9
Interfaz de salida:	m	j	m	0	k	j	k	n	j	k	n

Routing por vector distancia

- La principal virtud del algoritmo del vector distancia es su sencillez, los protocolos que lo utilizan consumen poca CPU y poca memoria en el router
- El vector distancia se utiliza en diversos protocolos de routing:
  - Internet estándar: RIP, **BGP**
  - Internet propietarios (Cisco): IGRP, **EIGRP**
  - Appletalk
  - DECNET (versiones antiguas)
  - IPX (versiones antiguas)

Problema de la cuenta a infinito

- El algoritmo del vector distancia es sencillo y eficiente, pero adolece de problemas debido a que el router maneja muy poca información y no conoce la ruta completa hacia el destino
- En determinadas situaciones se puede dar el problema conocido como 'cuenta a infinito' que deja inaccesibles a partes de la red

El problema de la cuenta a infinito

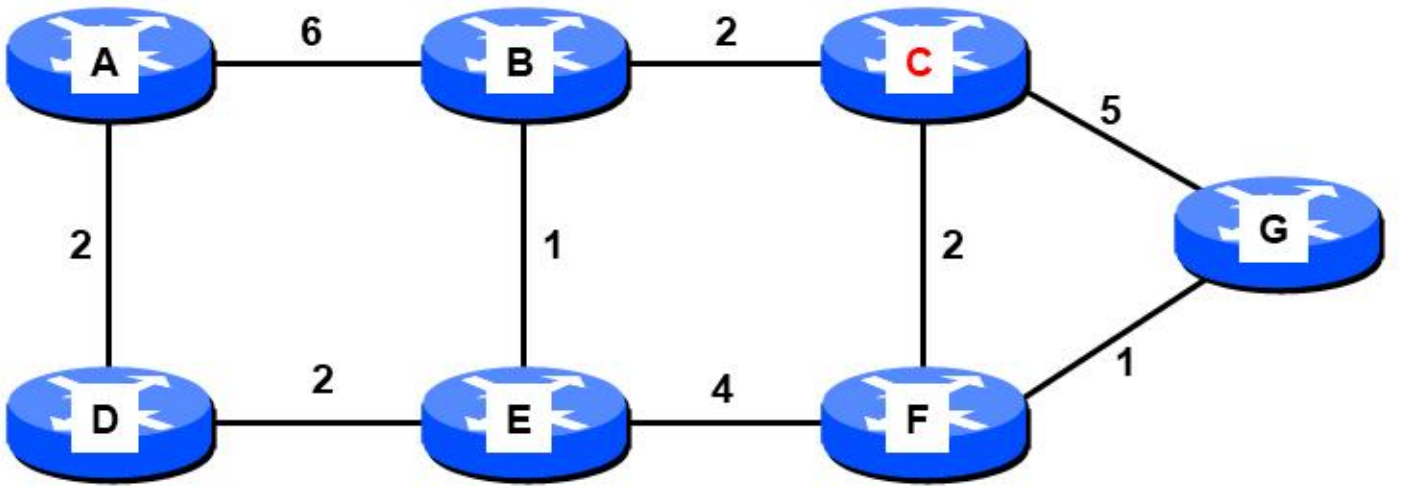


Distancias hacia A	-	∞	∞
A se enciende	0	∞	∞
	0	1	∞
	0	1	2
A se apaga	-	3	2
	-	3	4
	-	5	4
	-	5	6
	-	7	6
	-	7	8
	-	9	8
	.	.	.
	.	.	.
	.	.	.
	-	∞	∞

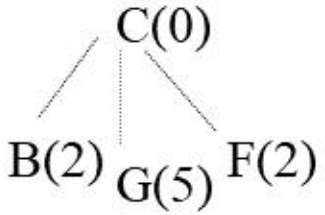
Solución a la cuenta a infinito

- Cuando la distancia se mide en número de saltos el problema de la cuenta a infinito es más fácil de resolver, pues basta con fijar un 'diámetro' máximo de la red
- Si se usan métricas basadas en el ancho de banda u otros parámetros que manejan un rango amplio de valores es mas difícil fijar un límite razonable
- Algunos protocolos incorporan modificaciones al algoritmo original dando información adicional para evitar el problema de la cuenta a infinito. Veremos un ejemplo cuando hablemos del protocolo BGP.

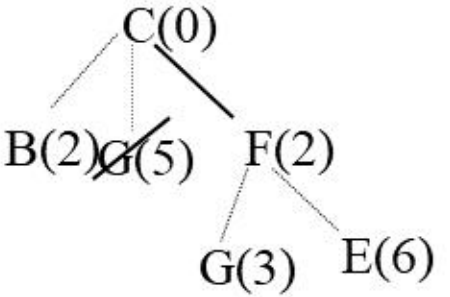
Algoritmo del estado del enlace (Dijkstra)



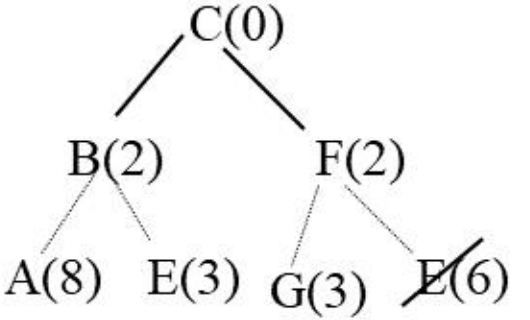
Link State Packets	A	B	C	D	E	F	G
	B/6	A/6	B/2	A/2	B/1	C/2	C/5
	D/2	C/2	F/2	E/2	D/2	E/4	F/1
		E/1	G/5		F/4	G/1	



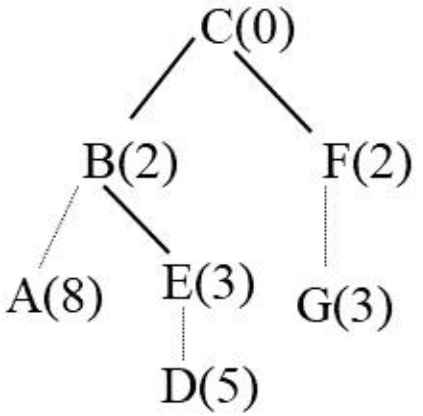
1: C examina su LSP.
Pone en el árbol a B, G y F



2: C examina el LSP de F.
Encuentra mejor camino hacia G,
corta rama peor

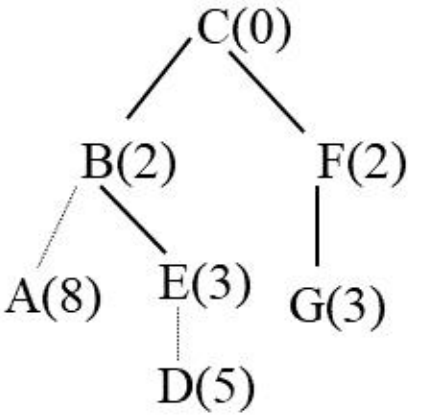


3: C examina el LSP de B. Mejor
camino hacia E, corta rama

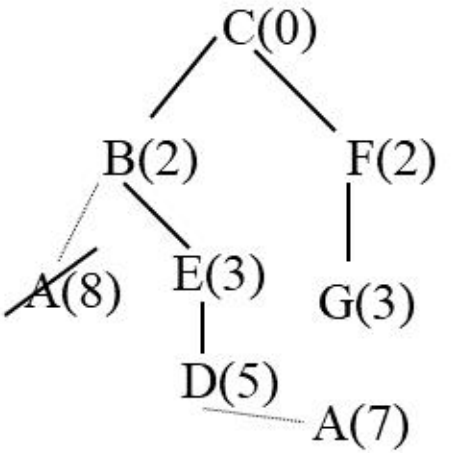


4: Examina LSP de E.
Pone a D en árbol.

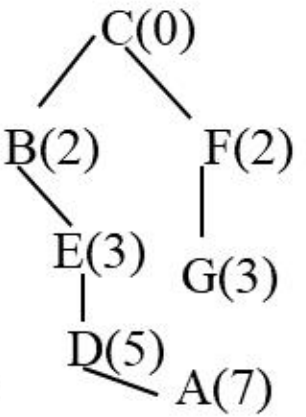
Algoritmo de Dijkstra



5: Examina LSP de G.
Sin cambios.



6: Examina LSP de D.
Pone A en el árbol



7: Examina LSP de A.
Sin cambios. No
quedan nodos, terminar

A	B	C	D	E	F	G
B/6	A/6	B/2	A/2	B/1	C/2	C/5
D/2	C/2	F/2	E/2	D/2	E/4	F/1
	E/1	G/5		F/4	G/1	

Árbol de rutas óptimas desde C para la red ejemplo

